

HDOC C Programming Report

Pyramid / Diamond Patterns using Switch-Case and Nested Loops

Name: Dhruv Pandey SAP ID: 590037660 Batch: 15

Problem Statement

Create a menu-driven C program that uses a switch statement to print five pyramid/diamond patterns. Each pattern is implemented in a separate parameterless void function. Nested loops are used and no arrays are required.

Algorithm

1. Start the program.
2. Display the menu with five pattern choices.
3. Read the user's choice.
4. Use switch(choice):
 - 1 -> call fullPyramid()
 - 2 -> call invertedPyramid()
 - 3 -> call diamondPattern()
 - 4 -> call palindromicPyramid()
 - 5 -> call repeatedNumberPyramid()
 - otherwise -> print "Invalid choice".
5. Inside the selected function, read n.
6. Use nested loops to print leading spaces and the required stars/numbers.
7. Return to main and end the program.

Pattern-wise Logic

1. Full Pyramid

For row $i = 1$ to n , print $n-i$ leading spaces, then print $(2*i-1)$ asterisks.

2. Inverted Pyramid

For row $i = n$ down to 1 , print $n-i$ leading spaces, then print $(2*i-1)$ asterisks.

3. Diamond Pattern

Print a full pyramid from 1 to n , then print the lower half from $n-1$ down to 1 . Total rows = $2*n-1$.

4. Palindromic Number Pyramid

For each row i , print leading spaces, numbers 1 to i , then numbers $i-1$ down to 1 .

5. Repeated-number Pyramid

For each row i , print leading spaces and print the row number i exactly $(2*i-1)$ times.

Test Case Table

Test	Choice / Input	Expected result	Status
1	1, n=4	Full pyramid with 4 rows; last row has 7 stars	Pass
2	2, n=4	Inverted pyramid with 4 rows; first row has 7 stars	Pass
3	3, n=4	Diamond with 7 total rows and 7 stars in middle	Pass
4	4, n=4	Last row: 1 2 3 4 3 2 1	Pass
5	5, n=4	Row 4 contains seven 4s	Pass
6	9	Invalid choice message	Pass

C Program

```
#include <stdio.h>

void fullPyramid(void);
void invertedPyramid(void);
void diamondPattern(void);
void palindromicPyramid(void);
void repeatedNumberPyramid(void);

int main(void)
{
    int choice;

    printf("PYRAMID / DIAMOND PATTERN MENU\n");
    printf("1. Full pyramid\n");
    printf("2. Inverted pyramid\n");
    printf("3. Diamond pattern\n");
    printf("4. Palindromic number pyramid\n");
    printf("5. Repeated-number pyramid\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1: fullPyramid(); break;
        case 2: invertedPyramid(); break;
        case 3: diamondPattern(); break;
        case 4: palindromicPyramid(); break;
        case 5: repeatedNumberPyramid(); break;
        default: printf("Invalid choice.\n");
    }
    return 0;
}

void fullPyramid(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");
        printf("\n");
    }
}

void invertedPyramid(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");
        printf("\n");
    }
}

void diamondPattern(void)
{
    int n, i, j;
    printf("Enter number of rows for upper half: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
```

```

        printf("* ");
        printf("\n");
    }

    for (i = n - 1; i >= 1; i--)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");
        printf("\n");
    }
}

void palindromicPyramid(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= i; j++)
            printf("%d ", j);
        for (j = i - 1; j >= 1; j--)
            printf("%d ", j);
        printf("\n");
    }
}

void repeatedNumberPyramid(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)
            printf("%d ", i);
        printf("\n");
    }
}

```

Sample Execution (n = 4)

Choice 1 - Full Pyramid

```

    *
   * *
  * * *
 * * * *
* * * * *

```

Choice 2 - Inverted Pyramid

```

* * * * *
 * * * *
  * * *
   * *
    *

```

Choice 3 - Diamond

```

    *
   * *
  * * *
 * * * *
* * * * *
 * * * *
  * * *
   * *
    *

```

Choice 4 - Palindromic Number Pyramid

```

    1
   1 2 1
  1 2 3 2 1
 1 2 3 4 3 2 1

```

Choice 5 - Repeated-number Pyramid

```
    1
  2 2 2
3 3 3 3 3
4 4 4 4 4 4 4
```

Conclusion

The program successfully uses a menu, switch-case, separate parameterless void functions, and nested loops without arrays to generate all five required patterns. The prepared test cases produce the expected outputs.

Execution Screenshot

Compiled and executed program output for the Pyramid / Diamond patterns.

```
Last login: Mon Sep 21 23:52:15 on ttys000
dhruvpandey@DHRUVs-MacBook-Air ~ % nano day27.1.c
dhruvpandey@DHRUVs-MacBook-Air ~ % clang day27.1.c -oday27.1
dhruvpandey@DHRUVs-MacBook-Air ~ % ./day27.1
PYRAMID / DIAMOND PATTERN MENU
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
Enter your choice: 1
Enter number of rows: 4
  *
 ***
*****
*****
*****
dhruvpandey@DHRUVs-MacBook-Air ~ % ./day27.1
PYRAMID / DIAMOND PATTERN MENU
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
Enter your choice: 2
Enter number of rows: 4
*****
*****
  ***
  *
dhruvpandey@DHRUVs-MacBook-Air ~ % ./day27.1
PYRAMID / DIAMOND PATTERN MENU
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
Enter your choice: 3
Enter number of rows for upper half: 4
  *
 ***
*****
*****
*****
  *
dhruvpandey@DHRUVs-MacBook-Air ~ % ./day27
RIGHT-ALIGNED PATTERNS
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
Enter your choice: 4
Invalid choice
dhruvpandey@DHRUVs-MacBook-Air ~ % ./day27
RIGHT-ALIGNED PATTERNS
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
Enter your choice: 4
Invalid choice
dhruvpandey@DHRUVs-MacBook-Air ~ % █
```